

Principles of Agile & Scrum Activities

Road to Success for Software Engineering

Reframe of Agile Principles

1) Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

→ The focus of project development/management is to provide consistently working software that makes our client's lives easier (the production of a piece of software for a client should not make their lives more difficult than it was before the project began).

2) Welcome changing requirements, even late development. Agile processes harness change for the customer's competitive advantage.

→ Changes to requirements should be welcomed with open arms, even in the later stages of development. Ultimately, projects are designed for the consumer—they should match the desires/requirements provided.

3) Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale

→ Working software products (that meet the needs of the client) should be delivered on a consistent, routine basis depending on the scope of the project; if projects can be delivered in a shorter time period, that is preferred.

4) Business people and developers must work together daily throughout the project.

→ Collaboration between business people/managers and developers throughout the project is necessary for success; this does not have to be daily but should be frequent enough that all participants of the project are on the same page.

5) Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

→ Build environments that are conducive to successful work; don't discourage employees when they hit roadblocks, instead provide them with pathways to success. This should be a collaborative effort.

6) The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

→ In-person, face-to-face contact (of quality) is the best way to ensure effective and consistent communication amongst team members throughout the development process. In-person communication circumvents potential miscommunications that

Principles of Agile & Scrum Activities

Road to Success for Software Engineering

are common in digital contexts.

7) Working software is the primary measure of progress.

→ Delivering working software is the marker of progress and long-term success of a project. Requirements and design are important, but having these artifacts without a working byproduct means nothing in terms of customer satisfaction.

8) Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

→ Set development/project routines that are sustainable for the team and do not break with fluctuation; sustainable practices should withstand time.

9) Continuous attention to technical excellence and good design enhances agility.

→ Adaptability and continual self-improvement are a virtue to agile teams (and in general). The more intentional the efforts put into gaining new skills the better the reward.

10) Simplicity--the art of maximizing the amount of work not done--is essential.

→ Be concise and pragmatic in all approaches to development. Do not complete work that is 1) not necessary; and 2) do not contribute to the project's end goal.

11) The best architectures, requirements, and designs emerge from self-organizing teams.

→ Allow teams the space to delegate tasks amongst themselves; team members who get to utilize their skills where they are best suited tend to do better and are more enthusiastic about the work they complete.

12) At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

→ Encouraging patterns of growth and new behaviors where needed amongst team members is vital to continual improvement. Refer to Principle 9.

Principles of Agile & Scrum Activities

Road to Success for Software Engineering

Scrum Activities (Term + Definitions) & Artifacts (in order of occurrence)

PBI Refinement

Essentially PBI refinement is the process of ensuring that Higher priority PBIs are sprintable, or close to it, as well as ensuring the PBIs are presented in the correct high to low priority order within the product backlog. Also remember that higher priority PBIs should have more information available so that the dev team can better understand those PBIs. Being sprintable means they are not super high level, however, that does not mean that high level PBIs are bad. It also means that they have acceptance criteria. At least in the scope of this project, it was uncommon to have high level PBIs that took time to break down. Also, in the scope of this project, it is key to try to incorporate feedback into the PBIs. If it feels as if your PBIs are a bit too “tasky” (meaning they leave no room for creativity to take place) then you need to change them to allow for creativity.

Initial Grooming

Initial grooming is basically the preparation phase of the project. The primary goal is to have at least 2 sprints ready to be executed but no more than 4. Another goal is to ensure that all core systems and documents are ready. This includes the definition of ready, definition of done, the VCS is setup, coding standards are established, etc. Plus, if this the first time you have worked with your team it provides an opportunity for you guys to begin the forming stage of the Tuckman's model. This basically consists of getting to know each other and figuring out how you guys will work with one another.

Sprint Planning

Sprint planning is geared towards the **current sprint**. This is the time when the dev team is supposed to break down the current PBIs into tasks, estimate them, and then pick which ones they want to do. This is mainly for the dev team's benefit. Our team started with a quick review of the current sprint's PBIs. This provides opportunities for the dev team to ask any clarifying questions before they jump into creating tasks out of the PBIs. From there they estimate them and each member picks a task they want to do. For more, view: [Sprint Planning Guide – Atlassian](#)

Daily Scrum/Stand-Up

The daily scrum serves as a check-in for the development team. This activity is timeboxed for 15 minutes. As the Scrum Master, keep this in mind:

“The [Scrum Master](#) ensures that the meeting happens, but the Developers are responsible for conducting the Daily Scrum. The Scrum Master teaches them to keep the Daily Scrum within the 15-minute time-box.”

Principles of Agile & Scrum Activities

Road to Success for Software Engineering

To be effective, **think of the following** (and expect to hear from the dev team) during the daily scrum:
“These are the things that I’ve done, the things I will do, and the obstacles I’ve faced”

Pertaining to obstacles, do not seek to resolve these during the 15-minute time-box. As the SM, if you’re aware of team members that can assist with the resolution of obstacles, put them in contact with one another to work together after the meeting.

Sprint/Backlog Grooming

This activity is basically a quick **look into the future** to ensure that everyone is on the same page and to see what the team is working towards. It also doubles as an opportunity to work with some of the upcoming PBIs if they need to be broken down. The idea is to always ensure that you have at least 2 sprints worth of PBIs ready at any given moment, not including the current sprint. This ensures that if the team was to chew through PBIs fast that there is more work to be done and to provide an idea of what the project is heading towards.

Sprint Review

Sprint review is essentially the validation step within a sprint. The dev team works with the product owner to validate that each of their completed tasks is related to a PBI in some way. Meaning that it meets the PBI’s (or PBIs’) acceptance criteria. Once everything is validated then a decision needs to be made on what is shippable and what is not. Whatever is shippable should move onto the sprint release.

Sprint Release

As the name suggests this is where all shippable features/functionality is put into “production”. Production in our team’s project is simply moving it to your main branch. We decided that there is no point for a production branch in this project because it is just going to stay the same as main since we were not actually delivering the product to anyone. Normally, you would have a separate branch for production.

Sprint Retrospective

Think about what went well and what can be improved upon within the Sprint processes. The retrospective occurs once within a sprint and is timeboxed based on the length of a sprint. For SE1’s intents and purposes, this is at a maximum of an hour per sprint (typically the est. is $\approx$ 1hr per week of Sprint work). The holistic purpose of this group meeting is to recognize the value of processes and correcting/sharing thoughts for change. This sets the stage for the next sprints. Review and Retrospective happen one after another- review focusing on the resulting product and retrospective focusing on the steps taken to get the resulting product (both steps focus on continued

Principles of Agile & Scrum Activities

Road to Success for Software Engineering

growth/success). The following article distinguishes the two processes from one another: [Review vs. Retrospective, or both? – Workpath](#)

Future Development/Goals for Continued Success & Takeaways

Remember that ideally another semester class will build on top of what your team does. Make it easier for them to be able to do so by commenting code, documenting designs, building readme.md files, how to deploy, etc.

Anything your team struggles with should be documented for the next class, so they don't continue to. Documenting and discussing roadblocks as they occur ensures there is time to rework processes and “redefine success” along the way.

Our takeaways were as follows:

- 1)** Set up the VCS and make it robust. GitHub will make your life easier in ways you would have never imagined.
- 2)** Bugs will probably happen, but that is OK. Bugs can be turned into PBIs rather than trying to incorporate them into the current sprint if they are too complicated to solve with the sustainable pace that the team has designated.
- 3)** This is not something that the whole team suffered from; however, it should be noted that this project is FOR LEARNING. If you are putting in your best effort always (remember, best effort first...if something goes wrong, be okay with that, and know how/when to readjust) and trying to learn then all will be well.

As one who suffers from anxiety, I know it is not easy to “just not worry.” But I can assure you that just putting forth whatever your best effort may be in each moment **will** lead to success. The efforts and intentions you put forth are the key to this project; if you know you're doing your best, you will step away with an experience of value.

- 4)** Treat the Daily Standup like a discussion rather than just a task that *needs* to get done. This builds camaraderie amongst the team and creates space for genuine acknowledgement of individual team member's contributions and the team-wide processes that are being used. This will also encourage “best effort first” behaviors amongst the team.

Principles of Agile & Scrum Activities

Road to Success for Software Engineering

Resources

This section details external and/or course resources that may be beneficial for review as a team member on the SE1 Scrum Project. They have been divided into three sections by team member or topic: For Product Owners, For Scrum Masters, & Scrum Processes/Activities-

For Product Owners:

KinserPedia - [Advice for New Product Owners GitHub Document](#)

For Scrum Masters:

KinserPedia – [Advice for New Scrum Masters GitHub Document](#)

Scrum Processes/Activities:

[Scrum.org](https://www.scrum.org/) – Hub for All Things Scrum

This document was drafted by Brody Laws, Product Owner (SE1, Spring '26) and Elijah Amador, Scrum Master (SE1, Spring '26) to assist future students and detail takeaways from the 4-Week Sprint Project in Software Engineering 1 @ ETSU.

[END OF REPORT]